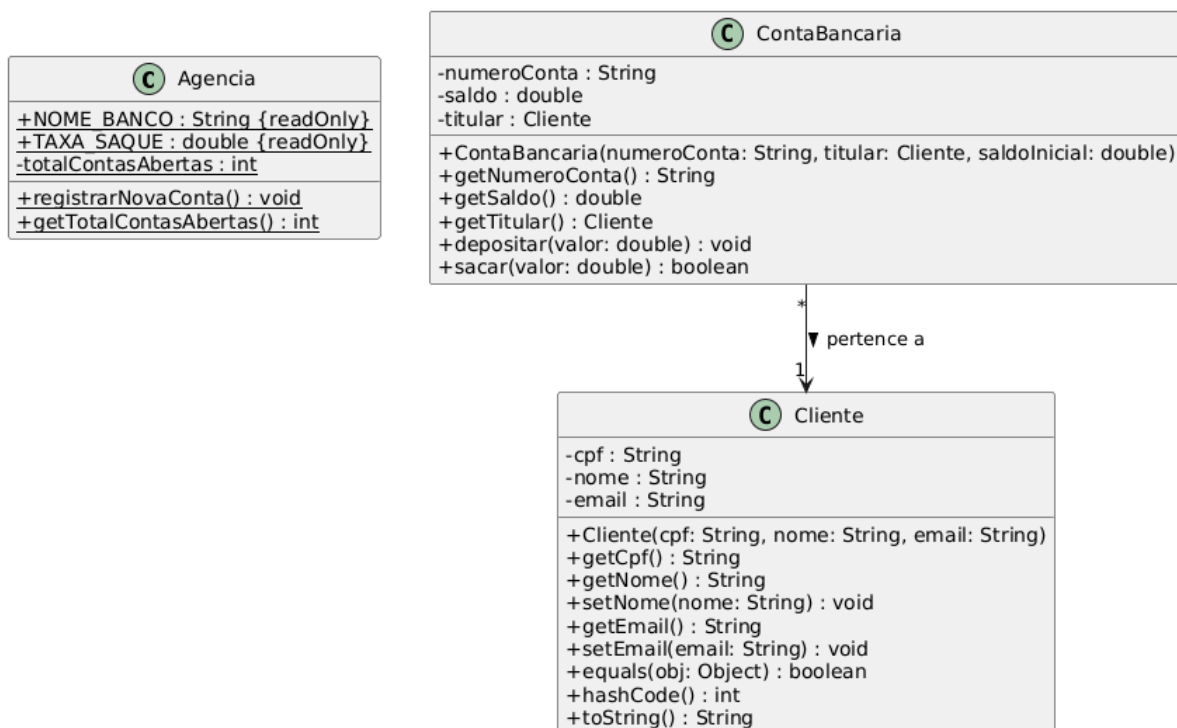


MISSÃO 04: Cofre Forte (Leitura Arquitetural)

Objetivo: No mercado de trabalho, o Arquiteto de Software não diz "crie um atributo private". Ele te entrega um Diagrama UML e um Card no Jira com as Regras de Negócio. A sua missão é transformar a especificação abaixo em código Java funcional, blindado e versionado.

PARTE 1: A Planta Arquitetural



PARTE 2: Documento de Regras de Negócio (Especificação)

Atenção, Desenvolvedor. O diagrama acima mostra a estrutura, mas o comportamento do sistema deve obedecer rigorosamente às regras abaixo. Uma falha aqui compromete a segurança do banco.

RN01 - Configurações Globais (Agencia):

- As constantes devem ser inicializadas direto na classe: NOME_BANCO recebe "JavaBank" e TAXA_SAQUE recebe 5.0.
- O método registrarNovaConta() deve apenas incrementar a variável totalContasAbertas.

RN02 - Identidade e Segurança do Cliente:

- Note no UML que o atributo cpf **não possui** método set. Um CPF jamais pode ser alterado após a criação do objeto.
- **Igualdade:** O sistema considera dois clientes IGUAIS se, e somente se, eles possuírem o mesmo cpf. Nome e e-mail devem ser ignorados na comparação (equals e hashCode).
- O método toString() do Cliente deve ocultar o CPF por segurança LGPD. Deve retornar algo no formato: Cliente: [Nome] | Contato: [Email].

RN03 - Movimentação Bancária (ContaBancaria):

- **Nascimento da Conta:** Toda vez que o construtor da ContaBancaria for acionado (uma conta for criada), ele deve obrigatoriamente chamar o método estático de registro da classe Agencia para contabilizar a nova conta.
- **O Cadeado do Saldo:** O saldo NUNCA pode ter um método *setter* (Note que não há setSaldo no UML).
- **Regra do Saque:** O saque só pode ocorrer se o saldo atual for maior ou igual ao valor solicitado **somado** à Agencia.TAXA_SAQUE. Em caso de sucesso, subtraia o total (valor + taxa) do saldo e retorne true. Caso contrário, não altere o saldo e retorne false.



PARTE 3: O Campo de Provas (MainTeste)

Crie um pacote br.com.banco.main, crie a classe MainTeste e construa um cenário que prove que sua arquitetura funciona:

1. Instancie dois Clientes diferentes, mas com o MESMO CPF.
2. Faça um if testando se eles são iguais usando o método correto de POO. Se o console não imprimir que eles são iguais, você falhou na RN02.
3. Instancie uma ContaBancaria com saldo de 50.0.
4. Tente realizar um saque de 50.0. O sistema deve negar (por causa da taxa). Imprima o retorno do método (deve ser false).
5. Imprima o total de contas abertas chamando a classe Agencia.

Seu código atendeu a todos os requisitos? Faça o git add, git commit e git push para o GitHub!